

DataFrameLib — Design Report

COP290 Assignment 4

Shoubhit Sanjay Wandile

Indian Institute of Technology Delhi

1 INTRODUCTION

In this assignment, I built `DataFrameLib`, a C++ library for processing tabular data. Since I had to rely strictly on Apache Arrow for the backend, I designed the library to support two execution paths: an Eager mode for immediate, Pandas-style data manipulation, and a Lazy mode that builds a computation plan. The real heavy lifting happens in the Query Optimizer, which intercepts the lazy DAG when `.collect()` is called and rewrites the tree to cut down on unnecessary memory usage and disk I/O before actual physical execution.

2 ARCHITECTURE

2.1 Layered Design

I split the engine into three distinct layers:

- **Storage Layer:** At the lowest level, the `Column` class wraps a `std::shared_ptr<arrow::Array>` and attaches a custom `DataType` tag. This keeps the data completely immutable.
- **Computation Layer:** `EagerDataFrame` handles actual execution. It stores columns in an `unordered_map` for $\mathcal{O}(1)$ lookups, but I also keep an `insertion_order` vector to make sure output files (like CSVs) maintain the correct schema order.
- **Plan Layer:** For deferred execution, `LazyDataFrame` holds a `std::shared_ptr<LogicalNode>`. I built subclasses for every operation (`ScanNode`, `FilterNode`, `JoinNode`, etc.). When `.collect()` is called, it triggers a recursive `evaluate()` chain from the bottom up.

2.2 Expression System (AST)

To handle user queries, I built an Abstract Syntax Tree (AST) rooted at the `Exp` class. To make the C++ API feel natural (like Python), I overloaded standard operators (`+`, `-`, `==`, `&`, etc.) using a thin `Expr` value-type wrapper. This allows users to type `col("age") + lit(5)` instead of manually juggling smart pointers. When evaluated, each node processes the current column context and returns a new materialized `Column`.

2.3 Type Safety

Getting Arrow types to play nicely was tricky, so I enforced type safety by routing everything through a custom `DataType` enum and a `promoteTypes()` helper. If an operation mixes `Int32` and `Float32`, it automatically upcasts to `Float64`. For invalid operations (like trying to do math on a `String` and an `Int`), the AST evaluation explicitly throws a runtime `std::invalid_argument` rather than silently failing. Nulls are strictly propagated—any operation on a null yields a null.

2.4 Lazy Execution & DAG Visualization

Because `LogicalNode` instances hold shared pointers to their parents, they naturally form a Directed Acyclic Graph (DAG). For the `explain()` method, I wrote a recursive function that traverses this DAG and generates Graphviz `.dot` syntax. I then use a system call to compile it to a `.png`. Crucially, `explain()` calls the Query Optimizer before generating the graph, so it shows the actual execution path, not just the raw user input.

3 QUERY OPTIMISATION

Before `collect()` executes, my `QueryOptimizer` runs four sequential passes over the DAG. Below are the details for each implemented rule.

3.1 CONSTANT FOLDING

Transformation: I wrote a recursive pass that looks for `BinaryOp` nodes where both children are `ConstantNodes`. If it finds one, it evaluates the math right then and there (during plan construction) and replaces the whole subtree with a single new `ConstantNode`.

Correctness: Since both inputs are literals, the result is completely data-independent. The math will evaluate to the exact same constant for every single row. Pre-computing it just skips the redundant work without altering the final values.

Concrete example:

```
Before: col("x") + (lit(2) * lit(3))
After:  col("x") + lit(6)
```

Performance benefit: This prevents the Arrow builder from executing millions of redundant multiplications. On a 100K-row dataset, folding one binary operator saves 100,000 float operations and the overhead of allocating intermediate arrays.

3.2 EXPRESSION SIMPLIFICATION

Transformation: After folding constants, I check `BinaryOps` for basic mathematical and boolean identities. For instance, I rewrite `x + 0` to just `x`, `x * 1` to `x`, and `x & true` to `x`.

Correctness: This relies on standard algebraic axioms. Type promotion handles any type safety, and because my engine propagates nulls, `x+0` yields null if `x` is null, which is the exact same behavior as just returning `x` directly.

Concrete example:

```
Before: (col("x") * lit(1)) + lit(0)
After:  col("x")
```

Performance benefit: This strips out entire layers of AST traversal. Instead of allocating memory and looping through arrays to multiply by 1, the engine just passes the original column pointer forward.

3.3 PREDICATE PUSHDOWN (FILTER SINKING)

Transformation: This pass pushes `FILTER` operations down the tree. If it hits a `SELECT`, they simply swap. If it hits a `WITH_COLUMN`, they swap as long as the filter doesn't rely on the newly generated column. If it hits a `JOIN`, I split the predicate by its `&` conjuncts. Clauses touching only the left table go to the left branch, right-table clauses go right, and mixed clauses stay above the join as residuals.

Correctness: For projections (`SELECT`), filtering first doesn't change the row logic, it just drops rows before the projection happens. For joins, a row only survives an inner join if the keys match and the filter passes. Pushing a left-only condition down just preemptively drops rows that would have failed the post-join filter anyway.

Concrete example:

```
Before: SCAN_L \ JOIN / SCAN_R -> FILTER(L.age > 30)
After:  [SCAN_L -> FILTER(L.age > 30)] \ JOIN / SCAN_R
```

Performance benefit: This is arguably the most important optimization. By filtering before the join, the engine builds and probes the hash table using a tiny fraction of the original data, changing an $\mathcal{O}(M \times L)$ bottleneck into a much cheaper operation.

3.4 LIMIT PUSHDOWN

Transformation: If the user calls `HEAD(n)`, I push that node below stateless operations like `SELECT` and `WITH_COLUMN`. I intentionally stop the pushdown if it hits stateful nodes like `SORT`, `FILTER`, or `JOIN`.

Correctness: A `SELECT` maps rows 1:1 without changing their order. Grabbing the top 10 rows and then projecting the columns is mathematically identical to projecting the columns and then taking the top 10. However, pushing `HEAD` below `SORT` is unsound (the top 10 of an unsorted file is entirely different from the top 10 of a sorted file), which is why the traversal halts there.

Concrete example:

```
Before: SCAN -> WITH_COLUMN("tax", col("salary") * 0.1) -> HEAD
        (10)
After:  SCAN -> HEAD(10) -> WITH_COLUMN("tax", col("salary") *
        0.1)
```

Performance benefit: Instead of doing the math to calculate "tax" on 1,000,000 rows only to discard 999,990 of them, the engine limits the scan to 10 rows and only computes the math 10 times.

3.5 PROJECTION PUSHDOWN

Transformation: I wrote a pass that figures out exactly which columns are needed by the downstream nodes (like filters or joins). As it traverses upwards, it pushes `SELECT` nodes down to the source. If a node outputs columns that are never used again, a new `SelectNode` is injected to immediately prune them.

Correctness: In relational algebra, if an operation only uses subset C of the available columns, proactively dropping any column outside of C has zero mathematical effect on the final result.

Concrete example:

```
Before: SCAN(file) -> FILTER(age > 30) -> SELECT(name, age)
After:  SCAN(file) -> SELECT(name, age) -> FILTER(age > 30)
```

Performance benefit: This is a massive save for Disk I/O. For wide datasets (like Parquet files with 100+ columns), pushing the projection directly to the `ScanNode` means the Arrow reader physically only loads the requested column chunks from the disk, saving gigabytes of unnecessary RAM allocation.

4 MEMORY MANAGEMENT

Because we used Apache Arrow, avoiding memory leaks was relatively straightforward by relying heavily on C++ RAII. All data buffers are strictly managed by `std::shared_ptr<arrow::Array>`. Instead of mutating data in place (which is dangerous in data pipelines), my Eager operations allocate new buffers via Arrow Builders and return entirely new `EagerDataFrame` objects.

I constructed the Lazy DAG using `std::shared_ptr<LogicalNode>` to ensure nodes are safely destructed when they go out of scope. I also heavily utilized `std::move` in constructors (like `JoinNode` and `FilterNode`) to avoid unnecessary reference counting overhead. I avoided using raw `new` and `delete` entirely across the codebase.

5 TESTING

I verified the implementation using the provided autograder framework. The engine passes all 79/79 sub-tests (100% correctness). This covers deep AST evaluation, chained query logic, edge cases like negative integer inputs, null propagation, and sort stability. I also ran local industrial stress tests (e.g., executing a `Read -> Filter -> Sort -> GroupBy -> Head` pipeline on a 50,000-row generated dataset) to verify that the query optimizer correctly rewires the DAG before execution without crashing or leaking memory.

6 CONCLUSION

Overall, `DataFrameLib` successfully implements a type-safe data engine. The eager mode performs reliably with Arrow's native arrays, while the Lazy mode effectively defers execution to optimize the workload. Implementing the rule-based optimizations—specifically Predicate and Projection pushdowns—proved to be highly effective at bypassing unnecessary computation, bringing the engine's logical performance much closer to real-world libraries like Polars.